

DocMind AI

Enterprise Document Intelligence & RAG Platform — Revision & Viva Preparation Guide

Author / Developer: Zoheb Qureshi | **Tech Stack:** Python, FastAPI, Next.js, PostgreSQL (pgvector), Supabase, LangChain, Google Gemini 3.6 Flash

Document Purpose: Comprehensive technical breakdown, database schema reference, key source code implementations, RAG logic walkthrough, and interview viva questions for DocMind AI.

1. Executive Project Summary & System Architecture

DocMind AI is an enterprise-grade AI Document Intelligence system that provides multi-tenant Retrieval-Augmented Generation (RAG). It enables organizations to upload complex PDF documents, automatically parse and vector-index them, and allow users to ask question-answer queries with precise source attribution (chunk index and cosine similarity scores).

Core Capabilities:

- **Multi-Tenancy & RBAC:** Strict organization-level data isolation with ADMIN and EMPLOYEE role differentiation.
- **Asynchronous Document Ingestion:** Non-blocking PDF uploading to Supabase Storage with FastAPI background worker tasks.
- **Advanced Vector Processing:** PDF loading with *PyPDFLoader*, text splitting using *RecursiveCharacterTextSplitter* (500 chars, 100 overlap), and embedding generation via *GoogleGenerativeAIEmbeddings* (3072 dimensions).
- **PostgreSQL + pgvector Integration:** Native vector search in PostgreSQL using cosine distance queries.
- **LangChain LCEL Pipeline:** Production RAG chain utilizing *RunnableParallel*, *RunnablePassthrough*, and *ChatGoogleGenerativeAI* (*gemini-3.6-flash*) with low thinking overhead.
- **Token & Cost Analytics:** Real-time AI audit logging tracking prompt/completion tokens, latency, and exact cost computation (\$0.75/1M input, \$3.75/1M output).
- **Automated Executive Insights:** AI-generated periodic summaries analyzing organization usage, token spend, employee activity, and performance trends.

Key Takeaway / Concept: *DocMind AI solves enterprise document retrieval by combining multi-tenant security with high-dimensional vector search (3072 dims) and zero-hallucination prompt constraints.*

Layer	Technologies Used	Key Responsibilities
-------	-------------------	----------------------

Backend Framework	FastAPI, Uvicorn, Pydantic	REST APIs, JWT authentication, dependency injection, background processing
Database & Vector	PostgreSQL, pgvector, SQLAlchemy 2.0, Alembic	Relational data (Orgs, Users, Docs, Logs) and 3072-dim vector similarity storage
AI & RAG Engine	LangChain LCEL, Gemini 3.6 Flash, Gemini Embeddings 001	Document chunking, query vectorization, cosine similarity retrieval, context-guided LLM inference
File Storage	Supabase Storage SDK	Cloud PDF storage scoped by organization and user directory paths
Frontend	Next.js 15 (App Router), TypeScript, Tailwind CSS	Responsive dashboard, document upload, interactive RAG chat, admin analytics UI

2. Database Design & Multi-Tenant Data Model

The relational database schema is designed using SQLAlchemy 2.0 type-annotated mapped columns and `pgvector` extensions. Every data model enforces strict tenant isolation via `organization\_id` foreign keys.

Model Name	Primary Keys & Indexes	Key Attributes & Features
Organization	id (PK, indexed)	name, created_at. Top-level tenant container.
User	id (PK), email (unique idx)	organization_id (FK), organization_user_id (scoped ID), name, password_hash (pwdlib), role (ADMIN/EMPLOYEE)
Document	id (PK), organization_id (idx)	filename, storage_key, uploaded_by (FK), status (UPLOADED/PROCESSING/READY)
DocumentChunk	id (PK), document_id (FK idx)	chunk_index, content (text), embedding (VECTOR(3072))
AIRequest	id (PK), org_id (idx), user_id (idx)	document_id, question, model, input_tokens, output_tokens, total_tokens, estimated_cost, latency, status

3. Crucial Code Implementations & Walkthrough

Below are the foundational code components that power DocMind AI. Study these snippets to explain the exact logic, class methods, and library calls during project review.

A. Database Schemas & Vector Column (app/models.py)

```

from datetime import datetime
from sqlalchemy import DateTime, ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from pgvector.sqlalchemy import VECTOR
from app.database import Base

class DocumentChunk(Base):
    __tablename__ = "document_chunks"

    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    document_id: Mapped[int] = mapped_column(ForeignKey("documents.id"), nullable=False, index=True)
    chunk_index: Mapped[int] = mapped_column(nullable=False)
    content: Mapped[str] = mapped_column(String, nullable=False)

    # 3072-Dimensional Vector Column for Gemini Embeddings
    embedding: Mapped[list[float] | None] = mapped_column(VECTOR(3072), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

class AIRequest(Base):
    __tablename__ = "ai_requests"

    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    organization_id: Mapped[int] = mapped_column(ForeignKey("organizations.id"), nullable=False, index=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    document_id: Mapped[int] = mapped_column(ForeignKey("documents.id"), nullable=False, index=True)
    question: Mapped[str] = mapped_column(String, nullable=False)
    model: Mapped[str] = mapped_column(String(100), nullable=False)
    input_tokens: Mapped[int] = mapped_column(default=0, nullable=False)
    output_tokens: Mapped[int] = mapped_column(default=0, nullable=False)
    total_tokens: Mapped[int] = mapped_column(default=0, nullable=False)
    estimated_cost: Mapped[float] = mapped_column(default=0.0, nullable=False)
    latency: Mapped[float] = mapped_column(default=0.0, nullable=False)

```

Key Takeaway / Concept: Notice the use of `pgvector's VECTOR(3072)` column type in SQLAlchemy mapped columns, specifically tailored for Google Gemini's high-resolution embeddings.

B. Asynchronous PDF Ingestion & Embedding Pipeline (app/documents_processor.py)

```
def process_document(document_id: int, storage_path: str):
    # 1. Download binary PDF from Supabase Storage
    pdf_bytes = supabase.storage.from_("Documents").download(storage_path)

    with tempfile.NamedTemporaryFile(suffix=".pdf", delete=False) as temp_file:
        temp_file.write(pdf_bytes)
        temp_path = temp_file.name

    db: Session = SessionLocal()
    try:
        # 2. Parse PDF pages via PyPDFLoader
        documents = load_pdf(temp_path)

        # 3. Chunk text (500 size, 100 overlap) via RecursiveCharacterTextSplitter
        chunks = split_documents(documents)
        chunk_texts = [chunk.page_content for chunk in chunks]

        # 4. Generate 3072-dim embeddings via GoogleGenerativeAIEmbeddings
        embedding_model = get_embedding_model()
        embeddings = embedding_model.embed_documents(chunk_texts)

        # 5. Save chunks + vector embeddings to PostgreSQL
        for index, (chunk, embedding) in enumerate(zip(chunks, embeddings)):
            document_chunk = DocumentChunk(
                document_id=document_id,
                chunk_index=index,
                content=chunk.page_content,
                embedding=embedding
            )
            db.add(document_chunk)

        # Update Document status to READY
        document = db.query(Document).filter(Document.id == document_id).first()
        if document:
            document.status = "READY"
        db.commit()
    finally:
        db.close()
        os.remove(temp_path)
```

C. Vector Cosine Similarity Search (app/langchain/retriever.py)

```
def retrieve_documents(
    query: str, document_id: int, organization_id: int, db: Session, top_k: int = 5
):
    # Enforce Tenant & Document Ownership Check
    document = db.query(DBDocument).filter(
        DBDocument.id == document_id,
        DBDocument.organization_id == organization_id
    ).first()
    if document is None:
        return []

    # Generate vector query embedding
    embedding_model = get_embedding_model()
    query_embedding = embedding_model.embed_query(query)

    # Perform Cosine Distance Vector Search in PostgreSQL using pgvector
    results = (
        db.query(
            DocumentChunk,
            (1 - DocumentChunk.embedding.cosine_distance(query_embedding)).label("similarity")
        )
        .filter(DocumentChunk.document_id == document_id)
        .order_by(DocumentChunk.embedding.cosine_distance(query_embedding))
        .limit(top_k)
        .all()
    )

    return [
        Document(
            page_content=chunk.content,
            metadata={"chunk_index": chunk.chunk_index, "similarity": float(similarity)}
        )
        for chunk, similarity in results
    ]
```

Key Takeaway / Concept: Cosine similarity is computed directly in SQL using `1 - cosine\_distance`. Query vectorization occurs in memory before executing the database filter.

D. LangChain LCEL RAG Pipeline (app/langchain/rag_chain.py)

```
def build_rag_chain(db, document_id: int, organization_id: int, top_k: int = 5):
    def retrieve(question: str):
        return retrieve_documents(
            query=question, document_id=document_id,
            organization_id=organization_id, db=db, top_k=top_k
        )

    retriever = RunnableLambda(retrieve)

    def format_documents(documents):
        return "\n\n".join(
            f"[Chunk {doc.metadata['chunk_index']}] \n {doc.page_content}"
            for doc in documents
        )

    # Complete LCEL Pipeline Execution
    rag_chain = (
        RunnableParallel(documents=retriever, question=RunnablePassthrough())
        .assign(context=lambda x: format_documents(x["documents"]))
        .assign(response=get_rag_chain())
        | RunnableLambda(build_result)
    )
    return rag_chain
```

E. Strict Prompt Guardrails & LLM Setup (app/llm.py)

```
def get_llm():
    api_key = os.getenv("GEMINI_API_KEY")
    return ChatGoogleGenerativeAI(
        model="gemini-3.6-flash",
        google_api_key=api_key,
        thinking_level="low"
    )

def get_rag_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", """You are an enterprise document assistant.

        Answer the user's question using ONLY the information provided in the context below.
        If the answer cannot be found in the context, say:
        "I could not find the answer in the provided documents."

        Do not make up information.

        Context:
        {context}"""),
        ("human", "{question}")
    ])
    return prompt | get_llm()
```

F. Real-Time Token Cost & Usage Analytics (app/analytics.py)

```
# Gemini 3.6 Flash Pricing Parameters
INPUT_PRICE_PER_MILLION = 0.75
OUTPUT_PRICE_PER_MILLION = 3.75

def calculate_cost(input_tokens: int, output_tokens: int) -> float:
    input_cost = (input_tokens / 1_000_000) * INPUT_PRICE_PER_MILLION
    output_cost = (output_tokens / 1_000_000) * OUTPUT_PRICE_PER_MILLION
    return input_cost + output_cost

def log_ai_request(db: Session, organization_id: int, user_id: int, document_id: int, question: str, result: dict):
    usage = result.get("usage_metadata") or {}
    response_meta = result.get("response_metadata") or {}

    input_tokens = usage.get("input_tokens", 0) or 0
    output_tokens = usage.get("output_tokens", 0) or 0
    total_tokens = usage.get("total_tokens", 0) or 0
    model = response_meta.get("model_name", "unknown")
    latency = result.get("latency", 0) or 0
    estimated_cost = calculate_cost(input_tokens, output_tokens)

    ai_request = AIRequest(
        organization_id=organization_id, user_id=user_id, document_id=document_id,
        question=question, model=model, input_tokens=input_tokens,
        output_tokens=output_tokens, total_tokens=total_tokens,
        estimated_cost=estimated_cost, latency=latency, status="SUCCESS"
    )
    db.add(ai_request)
    db.commit()
```

4. End-to-End System Execution Flow

Understanding how data moves through DocMind AI during document upload and RAG querying is critical for viva presentations.

Phase	Step-by-Step Flow	Security / Verification Guard
1. Upload Phase	1. User uploads PDF via Next.js UI. 2. FastAPI receives file, verifies <code>.pdf</code> extension and <code><= 10MB</code> limit. 3. File uploaded to Supabase Storage path: <code>{org_id}/{user_id}/{uuid}_{filename}</code>. 4. Database record created in <code>documents</code> with status <code>UPLOADED</code> -> <code>PROCESSING</code>. 5. <code>background_tasks.add_task(process_document, ...)</code> launched.	JWT Bearer Token validation (<code>get_current_user</code>). File size & format checks.
2. Ingestion Phase	1. Worker downloads PDF bytes to temp file. 2. <code>PyPDFLoader</code> extracts raw page texts. 3. <code>RecursiveCharacterTextSplitter</code> generates text chunks. 4. <code>GoogleGenerativeAIEmbeddings</code> calculates 3072-dim vectors. 5. <code>DocumentChunk</code> records inserted into PostgreSQL. 6. Document status updated to <code>READY</code>.	Database session rollback and Supabase storage cleanup on exception.
3. Query & RAG Phase	1. User submits question on <code>/ask</code> endpoint. 2. <code>retriever.py</code> verifies user's organization owns the target <code>document_id</code>. 3. Question converted into 3072-dim query embedding. 4. Cosine similarity query fetches top <code>k</code> chunks from PostgreSQL. 5. Chunks formatted into <code>{context}</code> prompt string. 6. Gemini 3.6 Flash generates answer strictly bounded by context. 7. Usage & cost logged into <code>ai_requests</code> table.	Organization-level data filtering in SQL query prevents cross-tenant data leakage.

5. Viva & Interview Questions Cheat Sheet

Here are the top questions evaluators and interviewers will ask about DocMind AI, along with crisp, impressive technical answers.

Q1: What is Retrieval-Augmented Generation (RAG) and why is it used in DocMind AI?

RAG is an AI architecture that combines Information Retrieval (vector database search) with Generative LLMs. Instead of relying on pre-trained internal knowledge, DocMind AI retrieves actual text chunks from uploaded corporate PDFs and injects them as prompt context. This eliminates hallucinations, keeps data current, and provides exact source chunk attribution.

Q2: How does DocMind AI handle Multi-Tenancy and Data Isolation?

Multi-tenancy is enforced at both application and database layers. Every JWT access token encodes ``organization_id``. Every SQL query (document retrieval, vector search, analytics) includes strict ``filter(Document.organization_id == user.organization_id)`` clauses. Storage paths in Supabase are also scoped by ``{organization_id}/{user_id}/``.

Q3: Why did you choose RecursiveCharacterTextSplitter with 500 chunk size and 100 overlap?

Recursive text splitting respects document hierarchy (paragraphs, sentences, spaces). A 500-character chunk size keeps context targeted and concise for accurate vector embeddings, while a 100-character overlap prevents key semantic information from being severed at chunk boundaries.

Q4: How does pgvector compare to standalone vector databases like Pinecone or ChromaDB?

Using pgvector inside PostgreSQL keeps relational application data (Users, Orgs, Audit logs) and high-dimensional vector embeddings in a single database. This allows atomic transactions, simplifies backup/restore, reduces infrastructure operational overhead, and enables JOINS between metadata and vectors in SQL.

Q5: What is LangChain Expression Language (LCEL) and why use RunnableParallel / RunnableLambda?

LCEL provides a declarative, production-ready way to compose custom chains without legacy monolithic classes. ``RunnableParallel`` allows executing document retrieval and prompt preparation concurrently, ``RunnablePassthrough`` forwards raw inputs seamlessly, and ``RunnableLambda`` converts standard Python functions into chain components.

Q6: How does the system track token usage and calculate LLM costs in real-time?

After every LLM call, Gemini returns ``usage_metadata`` containing ``input_tokens`` and ``output_tokens``. DocMind AI applies standard rate math (\$0.75 per million input tokens, \$3.75 per million output tokens) and logs exact metrics into the ``ai_requests`` table alongside request latency.

Q7: How does background document processing work in FastAPI?

FastAPI's ``BackgroundTasks`` executes ``process_document()`` in an asynchronous background thread pool right after returning HTTP 200/201 to the client. The client receives immediate confirmation with status ``PROCESSING``, while the backend downloads, chunks, embeds, and saves vectors without blocking the web request handler.

Key Takeaway / Concept: *Tip for Viva/Presentation: Highlight the combination of Enterprise Security (JWT + RBAC), Vector Math (3072-dim cosine distance), and Cost Intelligence (Token tracking) as your primary engineering achievements.*